CS210 Assignment 5

Author: Matthew Woo

Date: October 11, 2026

AI Disclosure

I used ChatGPT to learn how to implement each style of stack, queue, and deque. I also used ChatGPT to help me write out test cases.

Required Implementations

1. Implement a stack using an array-based representation.
2. Implement a stack using your own linked-list representation.
3. Implement a queue using an array-based representation.
4. Implement a queue using your own linked-list representation.
5. Implement a deque using an array-based representation.
6. Implement a deque using your own linked-list representation.
7. For each structure, demonstrate insertion and removal operations, empty handling, and a sequence of at least 10 operations.
8. Explain which operations are O(1) and which can become O(n) in your particular design.

Code

```
#include <iostream>
using namespace std;

/** AI Disclosure:
 * I used ChatGPT to learn how to implement each style of stack, queue, and
deque.
 * I also used ChatGPT to help me write out test cases.
 */

/** Implementation:
 * Stack:
 *  Pop();
 *  Push();
 *  Peek();
 *  IsEmpty();
 *  GetLength();
 * Queue:
 *  Enqueue();
```

```
 *  Dequeue();
 *  Peek();
 *  IsEmpty();
 *  GetLength();
 * Deque:
 *  PushFront();
 *  PushBack();
 *  PopFront();
 *  PopBack();
 *  PeekFront();
 *  PeekBack();
 *  IsEmpty();
 *  GetLength();
 */

class LinkedStack {
private:
    struct StackNode {
        int data;
        StackNode* next;

        // StackNode contructor
        StackNode(int newData, StackNode* nextNode) {
            data = newData;
            next = nextNode;
        }
    };
    StackNode* top = nullptr;
    int count = 0;

public:
    void Push(int newData) {
        top = new StackNode(newData, top);
        count++;
    }

    int Pop() {
        if (isEmpty()) {
            cout << "Stack is empty!" << endl;
            return -1;
        }
        StackNode* poppedNode = top;
        int poppedItem = top->data;
        top = top->next;
        delete poppedNode;
        count--;
        return poppedItem;
    }

    int Peek() {
        if (isEmpty()) {
            cout << "Stack is empty!" << endl;
            return -1;
        }
        return top->data;
```

```
    }

    bool isEmpty() {
        return top == nullptr;
    }

    int GetLength() {
        return count;
    }
};

class ArrayStack {
private:
    int items[20];
    int top = -1; // top=-1 so when top++, it starts at top=0

public:
    void Push(int newData) {
        if (top == 19) {
            cout << "Stack is full!" << endl;
            return; // end the runs
        }
        // if its not full then push the item in items[top]
        top++;
        items[top] = newData;
    }

    int Pop() {
        // Check if the stack is empty,
        // If empty then nothing to Pop
        if (top == -1) {
            cout << "Stack is empty!" << endl;
            return -1; // end the runs
        }

        int poppedItem = items[top];
        items[top] = 0; //clear the value in top
        top--;

        return poppedItem;
    }

    int Peek() {
        if (isEmpty()) {
            cout << "Stack is empty!" << endl;
            return -1;
        }
        return items[top];
    }

    bool isEmpty() {
        return top == -1;
    }

    int GetLength() {
```

```
        return (top + 1);
    }
};

class LinkedQueue {
private:
    struct QueueNode {
        int data;
        QueueNode* next;
        // QueueNode constructor
        QueueNode(int newData) {
            data = newData;
            next = nullptr;
        }
    };
    QueueNode* head = nullptr;
    QueueNode* tail = nullptr;
    int count = 0;

public:
    void Enqueue(int newData) {
        QueueNode* newNode = new QueueNode(newData);
        count++;
        if(tail == nullptr) { // if there is nothing in the queue
            head = newNode;
            tail = newNode;
        }
        else{
            tail->next = newNode; // connect current tail to newNode
            tail = newNode; // set tail to the newNode
        }
    }

    int Dequeue() {
        // Check if the queue is empty
        if (head == nullptr) {
            cout << "Queue is empty!" << endl;
            return -1;
        }

        QueueNode* removedNode = head;
        int removedItem = head->data; //Save the head to removedItem so it
can return later

        head = head->next; // change head to the element after head

        // If the queue becomes empty, reset tail
        if (head == nullptr) {
            tail = nullptr;
        }

        delete removedNode; //delete the head
        count--;
        return removedItem;
    }
```

```
    int Peek() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return -1;
        }
        return head->data;
    }

    bool isEmpty() {
        return head == nullptr;
    }

    int GetLength() {
        return count;
    }
};

class ArrayQueue {
private:
    int items[20];
    int count = 0;

public:
    void Enqueue(int newData) {
        if (count == 20) {
            cout << "Queue is full!" << endl;
            return;
        }
        items[count] = newData;
        count++;
    }

    int Dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return -1;
        }
        int removedItem = items[0];
        // Move all remaining items one position left, since the first one
left
        for (int i = 0; i < count - 1; i++) {
            items[i] = items[i + 1];
        }
        count--;
        items[count] = 0; // Clear the last position
        return removedItem;
    }

    int Peek() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return -1;
        }
        return items[0];
```

```
    }

    bool isEmpty() {
        return count == 0;
    }

    int GetLength() {
        return count;
    }
};

class LinkedDeque {
private:
    struct DequeNode {
        int data;
        DequeNode* next;
        DequeNode* prev;
        // DequeNode constructor
        DequeNode(int newData) {
            data = newData;
            next = nullptr;
            prev = nullptr;
        }
    };
    DequeNode* head = nullptr;
    DequeNode* tail = nullptr;
    int count = 0;

public:
    void PushFront(int newData) {
        DequeNode* newNode = new DequeNode(newData);
        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        }
        else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }
        count++;
    }

    void PushBack(int newData) {
        DequeNode* newNode = new DequeNode(newData);
        if (tail == nullptr) {
            head = newNode;
            tail = newNode;
        }
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
        count++;
```

```
    }

    int PopFront() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        DequeNode* removedNode = head;
        int removedItem = head->data;
        head = head->next;
        if (head == nullptr) {
            tail = nullptr;
        }
        else {
            head->prev = nullptr;
        }
        delete removedNode;
        count--;
        return removedItem;
    }

    int PopBack() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        DequeNode* removedNode = tail;
        int removedItem = tail->data;
        tail = tail->prev;
        if (tail == nullptr) {
            head = nullptr;
        }
        else {
            tail->next = nullptr;
        }
        delete removedNode;
        count--;
        return removedItem;
    }

    int PeekFront() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }

        return head->data;
    }

    int PeekBack() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        return tail->data;
```

```
    }

    bool isEmpty() {
        return head == nullptr;
    }

    int GetLength() {
        return count;
    }
};

class ArrayDeque {
private:
    int items[20];
    int count = 0;

public:
    void PushFront(int newData) { // ArrayQueue, but inserts at the front
        if (count == 20) {
            cout << "Deque is full!" << endl;
            return;
        }
        // Move items right to make space at the front
        for (int i = count; i > 0; i--) {
            items[i] = items[i - 1];
        }
        items[0] = newData;
        count++;
    }

    void PushBack(int newData) { // ArrayStack Push()
        if (count == 20) {
            cout << "Deque is full!" << endl;
            return;
        }
        items[count] = newData;
        count++;
    }

    int PopFront() { // ArrayQueue Dequeue()
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        int removedItem = items[0];

        // Move remaining items left
        for (int i = 0; i < count - 1; i++) {
            items[i] = items[i + 1];
        }
        count--;
        items[count] = 0; // Clear the last position
        return removedItem;
    }
```

```
    int PopBack() { // ArrayStack Pop()
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        count--;
        int removedItem = items[count];
        items[count] = 0;
        return removedItem;
    }

    int PeekFront() { // Queue Peek()
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        return items[0];
    }

    int PeekBack() { // Stack Peek()
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return -1;
        }
        return items[count - 1];
    }

    bool isEmpty() {
        return count == 0;
    }

    int GetLength() {
        return count;
    }
};

int main() {
    // cout true or false instead 1 and 0
    cout << boolalpha;
    int values[] = {10, 20, 30, 40, 50};
    int n = 5; //the size of the array

    // 1. STACK (Linked List and Array)
    cout << "=====STACK (LinkedList and Array)=====" << endl;
    LinkedStack linkedStack;
    ArrayStack arrayStack;
    // Push n elements into stack
    cout << "---Test Push for stack---" << endl;
    for (int i = 0; i < n; i++) {
        linkedStack.Push(values[i]);
        arrayStack.Push(values[i]);
        cout << "Push: " << values[i] << endl;
    }
    // Test Peek, return the top element without removing it
    cout << "---Test Peek for stack---" << endl;
```

```
    cout << "Linked Peek: " << linkedStack.Peek() << endl;
    cout << "Array Peek: " << arrayStack.Peek() << endl;
    // Test Pop, delete and return the first element of each loop
    cout << "---Test Pop for stack---" << endl;
    for (int i = 0; i < n; i++) {
        cout << "Linked Pop: " << linkedStack.Pop() << endl;
        cout << "Array Pop: " << arrayStack.Pop() << endl;
    }
    // Test GetLength and isEmpty, and print
    cout << "---Test GetLength and isEmpty---" << endl;
    cout << "Linked Size: " << linkedStack.GetLength() << endl;
    cout << "Array Size: " << arrayStack.GetLength() << endl;
    cout << "Linked Empty: " << linkedStack.isEmpty() << endl;
    cout << "Array Empty: " << arrayStack.isEmpty() << endl;
    // Test Empty Stack
    cout << "---Test Empty Stack---" << endl;
    linkedStack.Pop();
    arrayStack.Pop();


    // 2. QUEUE (Linked List and Array)
    cout << "\n=====QUEUE(LinkedList and Array)=====" << endl;
    LinkedQueue linkedQueue;
    ArrayQueue arrayQueue;
    // Enqueue n elements from the array to the queue
    cout << "---Test Enqueue---" << endl;
    for (int i = 0; i < n; i++) {
        linkedQueue.Enqueue(values[i]);
        arrayQueue.Enqueue(values[i]);
        cout << "Enqueue: " << values[i] << endl;
    }
    // Test Peek, return the first element without removing
    cout << "---Test Peek for queue---" << endl;
    cout << "Linked Peek: " << linkedQueue.Peek() << endl;
    cout << "Array Peek: " << arrayQueue.Peek() << endl;
    // Dequeue every elements in the queue
    cout << "---Test Deque---" << endl;
    for (int i = 0; i < n; i++) {
        cout << "Linked Dequeue: " << linkedQueue.Dequeue() << endl;
        cout << "Array Dequeue: " << arrayQueue.Dequeue() << endl;
    }
    // Test GetLength and isEmpty
    cout << "---Test GetLength and isEmpty---" << endl;
    cout << "Linked Size: " << linkedQueue.GetLength() << endl;
    cout << "Array Size: " << arrayQueue.GetLength() << endl;
    cout << "Linked Empty: " << linkedQueue.isEmpty() << endl;
    cout << "Array Empty: " << arrayQueue.isEmpty() << endl;
    // Test empty array cases
    cout << "---Test Empty Queue---" << endl;
    linkedQueue.Dequeue();
    arrayQueue.Dequeue();

    // 3. DEQUE (Linked List and Array)
    cout << "\n=====DEQUE(LinkedList and Array)=====" << endl;
    LinkedDeque linkedDeque;
```

```
    ArrayDeque arrayDeque;

    // Alternate between PushFront and PushBack
    cout << "---Test PushFront and PushBack for Deque---" << endl;
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) { // Test PushFront if i is even
            linkedDeque.PushFront(values[i]);
            arrayDeque.PushFront(values[i]);
            cout << "PushFront: " << values[i] << endl;
        }
        else { // If i is odd, then PushBack
            linkedDeque.PushBack(values[i]);
            arrayDeque.PushBack(values[i]);
            cout << "PushBack: " << values[i] << endl;
        }
    }
    // Test PeekFront and PeekBack
    cout << "---Test PeekFront and PeekBack for deque---" << endl;
    cout << "Linked PeekFront: " << linkedDeque.PeekFront() << endl;
    cout << "Linked PeekBack: " << linkedDeque.PeekBack() << endl;
    cout << "Array PeekFront: " << arrayDeque.PeekFront() << endl;
    cout << "Array PeekBack: " << arrayDeque.PeekBack() << endl;

    // Alternate between PopFront and PopBack
    cout << "---Test PopFront and PopBack for deque---" << endl;
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) { // PopFront if i is even
            cout << "Linked PopFront: " << linkedDeque.PopFront() << endl;
            cout << "Array PopFront: " << arrayDeque.PopFront() << endl;
        }
        else { // PopBack if i is odd
            cout << "Linked PopBack: " << linkedDeque.PopBack() << endl;
            cout << "Array PopBack: " << arrayDeque.PopBack() << endl;
        }
    }
    // Test GetLength and isEmpty
    cout << "---Test GetLength and isEmpty---" << endl;
    cout << "Linked Size: " << linkedDeque.GetLength() << endl;
    cout << "Array Size: " << arrayDeque.GetLength() << endl;
    cout << "Linked Empty: " << linkedDeque.isEmpty() << endl;
    cout << "Array Empty: " << arrayDeque.isEmpty() << endl;
    // Test PopFront and PopBack if the deque is empty
    cout << "---Test Empty Deque---" << endl;
    linkedDeque.PopFront();
    arrayDeque.PopBack();
    //End
    return 0;
}
```

Output

```
=====STACK (LinkedList and Array)=====
---Test Push for stack---
Push: 10
```

```
Push: 20
Push: 30
Push: 40
Push: 50
---Test Peek for stack---
Linked Peek: 50
Array Peek: 50
---Test Pop for stack---
Linked Pop: 50
Array Pop: 50
Linked Pop: 40
Array Pop: 40
Linked Pop: 30
Array Pop: 30
Linked Pop: 20
Array Pop: 20
Linked Pop: 10
Array Pop: 10
---Test GetLength and isEmpty---
Linked Size: 0
Array Size: 0
Linked Empty: true
Array Empty: true
---Test Empty Stack---
Stack is empty!
Stack is empty!

=====QUEUE(LinkedList and Array)=====
---Test Enqueue---
Enqueue: 10
Enqueue: 20
Enqueue: 30
Enqueue: 40
Enqueue: 50
---Test Peek for queue---
Linked Peek: 10
Array Peek: 10
---Test Deque---
Linked Dequeue: 10
Array Dequeue: 10
Linked Dequeue: 20
Array Dequeue: 20
Linked Dequeue: 30
Array Dequeue: 30
Linked Dequeue: 40
Array Dequeue: 40
Linked Dequeue: 50
Array Dequeue: 50
---Test GetLength and isEmpty---
Linked Size: 0
Array Size: 0
Linked Empty: true
Array Empty: true
---Test Empty Queue---
Queue is empty!
```

```
Queue is empty!

=====DEQUE(LinkedList and Array)=====
---Test PushFront and PushBack for Deque---
PushFront: 10
PushBack: 20
PushFront: 30
PushBack: 40
PushFront: 50
---Test PeekFront and PeekBack for deque---
Linked PeekFront: 50
Linked PeekBack: 40
Array PeekFront: 50
Array PeekBack: 40
---Test PopFront and PopBack for deque---
Linked PopFront: 50
Array PopFront: 50
Linked PopBack: 40
Array PopBack: 40
Linked PopFront: 30
Array PopFront: 30
Linked PopBack: 20
Array PopBack: 20
Linked PopFront: 10
Array PopFront: 10
---Test GetLength and isEmpty---
Linked Size: 0
Array Size: 0
Linked Empty: true
Array Empty: true
---Test Empty Deque---
Deque is empty!
Deque is empty!
```

Design decision

I implemented array and linked-list style stack, queue, and deque. I also tested different operations and empty cases.

Explanation, time/space complexity discussion

LinkedStack is O(1) because Push() and Pop() only change the top pointer; it doesn't need to go through the whole linked list. Peek() doesn't change anything.

ArrayStack is O(1) because Push() and Pop() only increase or decrease the top index and access one array element. Peek() doesn't change anything.

LinkedQueue is O(1) because Enqueue() and Dequeue() we use head and tail pointers to add or remove nodes directly without traversing the list.

ArrayQueue are both O(1) and O(n). Enqueue is O(1) because it only adds an item at the end of the array. Dequeue() is O(n) because it removes the top element and shifts the whole queue left by 1.

LinkedDeque is O(1). It uses a doubly linked list to store head, tail, prev, and next pointers. It can easily to add or remove items without traversing the list.

ArrayDeque are O(1) and O(n). PushBack() and PopBack() are O(1) because it only changes the last element of the array. PushFront() and PopFront() are O(n) because they need to shift the elements left or right by 1 when pushing and popping.